



Introducció a l' algorísmica

Algorísmica

Grau d'Enginyeria Informàtica

Mireia Ribera, ribera@ub.edu

Introducció a l'algorísmica: Conceptes clau

1

Què és un algorisme? Entrada, sortida. Eficiència i correcció

2

Què és un error i quins tipus hi ha

3

Llenguatges de programació: sintaxi i semàntica

Què és un algorisme? Entrada, sortida. Eficiència i correcció

Un algorisme és qualsevol **procediment computacional** que pren un (o una sèrie) de dades/valors com a **entrada** i genera alguna dada/valor (o sèrie de dades/valors) com a **sortida**.

- Els algorismes són les **idees/estratègies** que hi ha darrere els programes per resoldre un determinat problema.
- Els algorismes són **independents del llenguatge** en que estan escrits. El mateix algorisme escrit en dos llenguatges diferents pot tenir una aparença superficial molt diferent.
- Els algorismes **sí que depenen de la representació de les dades**.
- Els algorismes interessants són els que resolen problemes generals.

Exemple: Multiplicació d'enters

Quan definim un problema (per exemple, la multiplicació de dos nombres) a resoldre mitjançant un algorisme, ens cal especificar tres elements:

- L'entrada: En el cas de la multiplicació l'entrada seran dos nombres enters, a i b .
- Quina sortida esperem: Un nombre c tal que $c = a * b$.
- Quins requeriments imposem: La solució ha de ser **correcta** i **eficient**.

Hi ha **diversos algorismes** que resolen aquest problema.

El que heu vist a l'escola és un d'ells, però hi ha un algorisme “producte rus” que també la resol.

Exemple: Multiplicació d'enters – Algorisme Rus

En aquest algorisme, a i b van canviant de valor fins arribar a la solució. En primer lloc $a = a$ dividit per 2, si el mòdul de la divisió és 1, el resultat s'incrementa en b. $b = b$ multiplicat per 2 .I així fins que el primer operand és 0. Per exemple amb $47 * 12$, obtenim 564

a = 47	b = 12	resultat
47 Dividim //2 mentre $a > 0$	b = 12 Multipliquem x2	
$a = 47 // 2 = 23$; mòdul 1	$b = 12 * 2 = 24$	12 Si $a \% 2 == 1$ sumem b
$a = 23 // 2 = 11$; mòdul 1	$b = 24 * 2 = 48$	36 sumem
$a = 11 // 2 = 5$; mòdul 1	$b = 48 * 2 = 96$	84 sumem
$a = 5 // 2 = 2$; mòdul 1	$b = 96 * 2 = 192$	180 sumem
$a = 2 // 2 = 1$; mòdul 0	$b = 192 * 2 = 384$	180 no sumem
$a = 1 // 2 = 0$; mòdul 1	$b = 384 * 2 = 768$	564 sumem

Algorisme Rus – Codificació en Python

```
def producte_rus(a: int, b: int) -> int:
    """ Multiplica dos enters a i b utilitzant l'algorisme del producte rus. Retorna
    el resultat d' a * b. """
    resultat = 0
    while a > 0:
        # Si a és senar, afegeix b al resultat
        if a % 2 == 1:
            resultat += b
        # Divideix a per 2 (divisió entera) i multiplica b per 2
        a //= 2
        b *= 2
    return resultat
```

Correcció

Un algorisme és **correcte** si podem demostrar (matemàticament) que retorna la sortida desitjada per qualsevol entrada legal.

Demostrar la correcció a vegades és fàcil, però sovint és difícil o impossible.

Per exemple, en el cas de l'algorisme rus, si considerem només positius i els representem en binari, tenim que $a = \sum_{i=0}^k \alpha_i 2^i$ amb $\alpha_i \in \{0,1\}$ i llavors $a \cdot b = \sum_{i=0}^k \alpha_i (2^i \cdot b)$. L'algorisme recorre exactament els valors $2^i b$ duplicant b i suma al resultat aquells termes corresponents a $\alpha_i = 1$ quan a té el bit i en 1.

A l'exemple, amb $a=47$, la seva representació binària és 1011111

Exemple: Algorisme Rus - Explicació

En aquest algorisme, es divideix el primer operand per 2, si el mòdul és 1, se suma el segon operand al resultat. El segon operand es multiplica per 2. I així fins que el primer operand és 0. Per exemple amb $47 * 12$, obtenim 564

a = 47 = 1011111	b	resultat	
47 Dividim //2	12 Multipliquem x2		0, 1
23	24	12 Si $a\%2==1$ sumem b	1, 1
11	48	36	2, 1
5	96	84	3, 1
2	192	180	4, 1
1	384	180	5, 0
0 Repetim si $a>0$	768	564	6, 1

Eficiència

Un algorisme és eficient si es fa amb el mínim nombre de recursos de temps – cicles de rellotge -, i d'espai – memòria -.

Fer servir algorismes eficients és sempre convenient i a vegades és una necessitat!

Per exemple, en el cas de l'algorisme rus, les operacions de multiplicació per 2 o de divisió per 2 en binari són molt eficients, ja que només signifiquen moure els bits a l'esquerra o a la dreta una posició (veure exemple). Mirar si un nombre és senar només consisteix a mirar si el seu últim bit és un 1, i sumar, com veurem més endavant també és una operació eficient.

Exemple:

multiplicació binària $110 \times 2 = 1100$, divisió entera binària $110 // 2 = 10 = 11$, $111 // 2 = 11$

Algorismes i ordinadors

Un ordinador només fa dues coses:

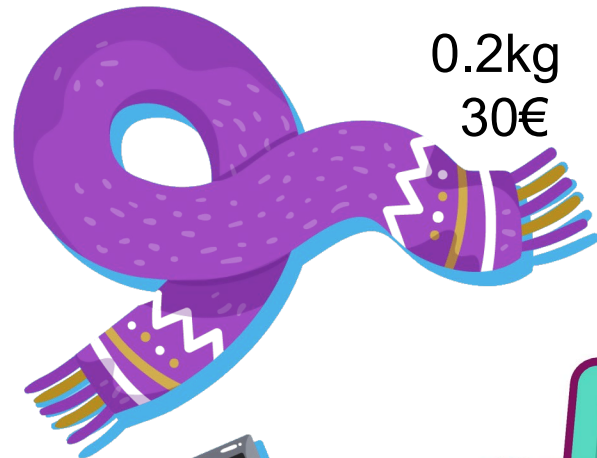
- **Calcular** (combinar dades per obtenir altres dades)
- **Emmagatzemar** (llegir i escriure a una memòria).

Un ordinador de sobretaula actual, amb 8 nuclis i 4GHz, fa fins a 256.000.000.000 càlculs per segon. Un ordinador amb 32GB de RAM pot emmagatzemar fins a 256.000.000.000 bits a memòria. I amb un disc dur d'1 TB pot emmagatzemar 8.000.000.000.000 bits.

En aquest curs veurem algorismes basats en un ordinador de sobretaula convencional. No veurem algorismes:

- Basats en càlcul paral·lel ni distribuït entre diversos ordinadors
- Basats en arquitectures no convencionals (quàntiques)

El problema de la motxilla



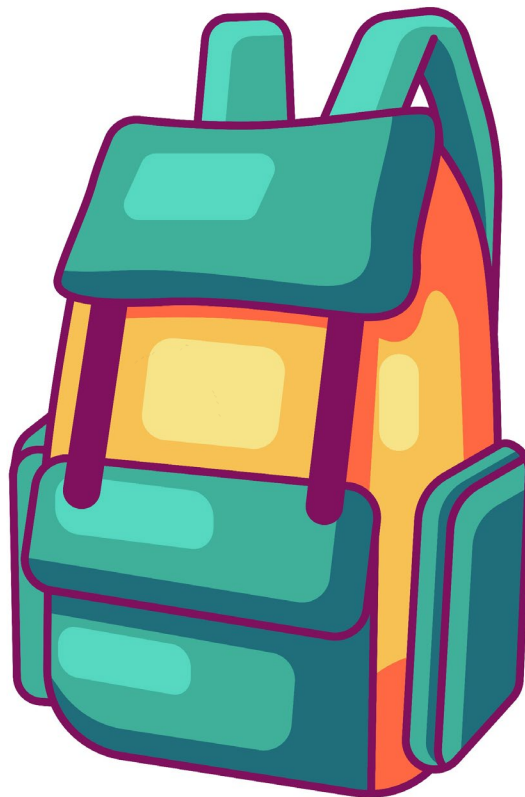
0.2kg
30€



2kg
150€



0.4kg
300€



0.5kg
50€

2.3kg

Definició del problema

La motxilla pot portar un pes màxim de 2.3kg

Disposem d'un seguit d'objectes que hi volem posar, cadascun amb un pes w_{obj_i} i amb un valor v_{obj_i} . Exemple: la bufanda pesa 0.2kg (w_{obj}) i té un valor de 30€ (v_{obj}).

Volem triar aquells elements que maximitzin el valor de la motxilla un cop carregada.

Aquest és un problema genèric per a portafolis financers, càrregues de vaixells o avions, tall de materials o emplenat de contenidors.

Quina és l'*entrada*? Quina la *sortida esperada*? Com es defineix *correcte*? i *eficient*?

Solució 1

Anem posant els objectes tal i com ens arriben fins a emplenar la motxilla

És correcte? és eficient?

Solució 2

Considerem totes les combinacions possibles d'elements i agafem la combinació més valuosa.

És correcte? és eficient?

Considerem totes les combinacions possibles d'elements. *Eliminem les que pesen més del màxim establert.* Triem la combinació més valuosa.

És correcte? és eficient?

Càlcul



2.3kg

Botes 2kg 150€	Mòbil 0.4kg 300€	Bufanda 0.2Kg 30€	Crema 0.5Kg 50€	Pes	Valor
0	0	0	1	0.5kg	50
0	0	1	0	0.2kg	30
0	0	1	1	0.7kg	80
0	1	0	0	0.4kg	300
0	1	0	1	0.9kg	350
0	1	1	0	0.6kg	330
0	1	1	1	1.1kg	380
1	0	0	0	2kg	150
1	0	0	1	2.5kg	200
1	0	1	0	2.2kg	180
1	0	1	1	2.7kg	230
1	1	0	0	2.4kg	450
1	1	0	1	2.9kg	500
1	1	1	0	2.6kg	480
1	1	1	1	3.1kg	530

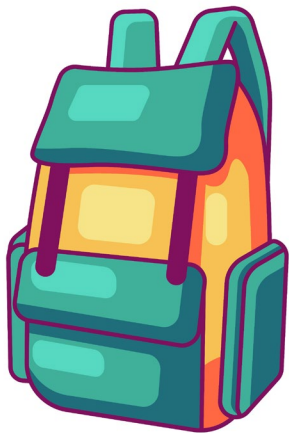
Eliminació



2.3kg

Botes 2kg 150€	Mòbil 0.4kg 300€	Bufanda 0.2Kg 30€	Crema 0.5Kg 50€	Pes	Valor
0	0	0	1	0.5kg	50
0	0	1	0	0.2kg	30
0	0	1	1	0.7kg	80
0	1	0	0	0.4kg	300
0	1	0	1	0.9kg	350
0	1	1	0	0.6kg	330
0	1	1	1	1.1kg	380
1	0	0	0	2kg	150
1	0	0	1	2.5kg	200
1	0	1	0	2.2kg	180
1	0	1	1	2.7kg	230
1	1	0	0	2.4kg	450
1	1	0	1	2.9kg	500
1	1	1	0	2.6kg	480
1	1	1	1	3.1kg	530

Tria final



2.3kg

Botes 2kg 150€	Mòbil 0.4kg 300€	Bufanda 0.2Kg 30€	Crema 0.5Kg 50€	Pes	Valor
0	0	0	1	0.5kg	50
0	0	1	0	0.2kg	30
0	0	1	1	0.7kg	80
0	1	0	0	0.4kg	300
0	1	0	1	0.9kg	350
0	1	1	0	0.6kg	330
0	1	1	1	1.1kg	380
1	0	0	0	2kg	150
1	0	0	1	2.5kg	200
1	0	1	0	2.2kg	180
1	0	1	1	2.7kg	230
1	1	0	0	2.4kg	450
1	1	0	1	2.9kg	500
1	1	1	0	2.6kg	480
1	1	1	1	3.1kg	530

Com expresseu els algorismes

Amb llenguatges de programació.

Un llenguatge de programació es defineix per unes primitives (símbols), una sintaxi (regles de combinació de símbols), una semàntica estàtica (combinacions de símbols amb significat) i una semàntica (el significat que nosaltres volem donar a l'algorisme).

Ex. A Python `def` és un símbol, la sintaxi de `print` és posar l'argument entre parèntesi, no podem fer `"hola" + 4` (semàntica estàtica), i l'algorisme del producte rus resol la multiplicació.

Com escriure bé

Quan programem, pot ser que **cometem errors**, per això les eines de programació ens ajudaran a detectar errors.

Els **errors inicials més comuns seran errors sintàctics**, consistents en no escriure bé les instruccions, no posar els espais que toquen, no posar parèntesi... Aquests **són els errors més fàcils de detectar i corregir pels entorns de programació**.

A vegades també cometrem **errors de semàntica estàtica**, combinant variables o fent operacions que no són correctes. **Alguns d'aquests errors també es poden detectar automàticament**.

Errors

Si no hi ha errors sintàctics ni de semàntica estàtica, **el programa s'executarà** i farà alguna cosa, però **no necessàriament el que volem**, estarem cometent **errors de semàntica**. Si ens ajudem de la IA per programar, ens pot proposar algorismes amb errors semàntics també.

Els errors semàntics són molt difícils de detectar de manera automàtica:

La correctesa semàntica del codi és **responsabilitat del o de la programador/a**.

```
def suma(a:int, b:int)->int:  
    """ Aquest programa no fa el què ha de fer! """  
    return a - b
```

Gràcies i recapitulació



UNIVERSITAT DE
BARCELONA

Recapitulació

-  Els algorismes són solucions correctes i eficients a problemes generals
-  Els errors semàntics són els més difícils de detectar i són responsabilitat del/de la programador/a.
-  Aquest curs programarem amb Python i amb Jupyter Notebooks
-  Cal aprendre les estructures bàsiques del llenguatge